A smaller checked presentation of intuitionistic primitive recursive arithmetic

A measured improvement and an exact criterion for closure

Prepared for Vladimir Reshetnikov
with ChatGPT

11 November 2026

Abstract

The current ConciseIPRA directory contains a highly compressed, formally studied presentation of full first-order intuitionistic primitive recursive arithmetic. Its six incoming research archives must also be included in the comparison: they already attain 257 significant tokens, 279 AST nodes, and 522 normalized Python bytes, in different complete programs.

This investigation obtains a smaller checked presentation by composing two existing ideas: the latest guarded-source checker and the scalar neutral resource established in the reversal package. The resulting byte profile has 521 normalized bytes, 263 significant tokens, 300 AST nodes, and 327 gzip-9 bytes. A separate profile has 278 AST nodes. The 257-token frontier is unchanged. The mathematical public theory and its bi-interpretation with iPRA are inherited without alteration.

The improvement is implementation-level. A new, simpler mathematical axiomatization was not obtained. No absolute minimum or significant arithmetic breakthrough is claimed. The report specifies the theory, the inherited equivalence boundary, the exact code change, and 258 successful process expectations. It also proves a sharp criterion for when conditional and progressive equation closures coincide, explaining the limitation of several tempting rule deletions. The archive includes the full inherited mathematical report, exact comparison sources, executable checkers, certificates, measurements, and reproduction commands.

Contents

1 What the comparison actually establishes **3**

2 The target theory and the inherited presentation **3**

3 Why the inherited theory has the required strength **5**

4 The neutral-resource reduction **7**

5 Measurements and executable evidence **9**

6 An exact criterion for conditional and progressive closure **9**

7 Further research questions **12**

8 Reproduction, attribution, and status **13**

1 What the comparison actually establishes

The inspected repository is Vladimir Reshetnikov's ProveIt, at commit

`a4e5e96c456303cf231eab2aeeb3291d8889833f`.

The target is

`Logic/PrimitiveRecursiveArithmetic/ConciseIPRA`.

Its README describes the scalar native calculus, 28 research packages, and the Lean equivalence with full first-order iPRA [1]. Six additional archives in its `docs/incoming` directory were retrieved and inspected. Their own comparison snapshot is the older commit `1517d9d79ce33f631e0f82960c849427cbba23bf`. The two snapshots are not conflated.

The relevant incoming baseline is `ipra_guarded_source_2026-11-11`, whose separately optimized complete modules attain

257 tokens, 279 AST nodes, 522 bytes.

The README's 259/283/526 figures are therefore not the strongest available comparison. Independent remeasurement of the incoming modules reproduced their reported figures. The complete audit is included in `reference/incoming_metrics_audit.json`.

The change made here is narrow:

```
# Byte profile                  # AST profile
g=[[]]       -> g=[0]            g=([],)       -> g=(0,)
```

The expression is a default retained context of administrative truth. The associated neutral instruction majors are encoded as `0`. This representation was already justified in the incoming reversal report [3]. Applying it to the newer guarded-source programs gives a further one-byte and one-node improvement. Neither ingredient is claimed as a new mathematical idea.

The requested broader objective remains open in this investigation: removing a genuine primitive mathematical operation or inference pattern, while retaining the required bi-interpretation and reducing the size of a straightforward complete checker. The files provide a concrete verified increment and the reasoning needed to assess its modest scope.

2 The target theory and the inherited presentation

2.1 Meaning of iPRA

The target is first-order intuitionistic logic with equality, one numerical sort, zero, successor, and one function symbol for each fixed primitive recursive definition. Such definitions are generated from zero, successor, projections, composition, and primitive recursion. The defining equations are included for each definition. For example, a primitive recursion description introduces a symbol f with

$$f(0,\vec{a}) = g(\vec{a}), \qquad f(Sn,\vec{a}) = h(n, f(n,\vec{a}),\vec{a}).$$

The nonlogical axioms also include successor injectivity, $Sx \neq 0$, and induction for quantifier-free formulas, with numerical parameters. In particular the induction schema is

$$A(0,\vec{a}) \wedge \forall n\big(A(n,\vec{a}) \to A(Sn,\vec{a})\big) \;\to\; \forall n\, A(n,\vec{a}),$$

where A is quantifier-free. There is no induction over arbitrary first-order formulas. This is the convention used by the repository's deeply embedded `PR` theory and its equivalence theorems.

2.2 Numerical terms

The presented calculus uses the scalar numerical syntax

$$t ::= x \mid 0 \mid St \mid I(n, a; u.h).$$

Only u is bound in h. The counter n and initializer a are outside that binder; h can contain fixed parameters. Bound variables are renamed to avoid capture. Set $1 = S0$.

There are exactly two computation schemas, in every retained context:

$$I(0, a; u.h) = a, \tag{I0}$$
$$I(Sn, a; u.h) = h[I(n, a; u.h)/u]. \tag{IS}$$

Each finite iterator description denotes a primitive recursive function. No numerical variable ranges over descriptions and no uniform evaluator for all primitive recursive functions is assumed.

2.3 Public formulas and auxiliary proof formulas

Public formulas are equations and paired formulas:

$$A ::= (s = t) \mid \mathsf{D}_{x,y} B.$$

At an atomic matrix beneath D, the canonical public grammar requires

$$B(x, y) \equiv I(y, 0; u.h) = I(y, 0; u.k). \tag{Guard}$$

The update terms may have parameters, including x, y, as well as their state variable. Compound matrices satisfy the same grammar recursively. An equation at the top level is unrestricted.

Proofs may additionally use auxiliary formulas

$$F ::= (s = t) \mid \mathsf{Q}_y F \mid \mathsf{D}_{x,y} F$$

with unrestricted atomic matrices. Thus the public formula language is smaller than the auxiliary proof language. This boundary is part of the presentation. The full auxiliary language is not asserted to have the same formula round trips as the public theory.

The ordinary public interpretation keeps numerical equality and uses

$$(\mathsf{D}_{x,y} B)^\circ := \exists x (B^\circ(x, 0) \to \forall y\, B^\circ(x, y)). \tag{D}$$

The designated contradiction is $0 = 1$. An administrative scalar zero used by the checker is *not* this contradiction: the contradiction is a binary equation node.

2.4 Every primitive inference pattern

Contexts are finite reusable lists of auxiliary formulas. Besides (I0) and (IS), the calculus has the following four patterns. They are inherited from guarded-source [2].

Hypothesis.

$$\frac{}{\Gamma \vdash F} \quad (F \in \Gamma). \tag{Hyp}$$

Elimination selected by a numerical source. For a fixed auxiliary template $F(v)$ set

$$M(s,t;F) = \begin{cases} \mathsf{Q}_v F(v), & s \text{ is literally } 0, \\ s = t, & s \text{ is not literally } 0. \end{cases}$$

Then

$$\frac{\Gamma \vdash M(s,t;F) \qquad \Gamma \vdash F(s)}{\Gamma \vdash F(t)}. \tag{SE}$$

The source restriction is syntactic. A variable or an iterator that denotes zero still belongs to the second branch.

Paired elimination.

$$\frac{\Gamma \vdash \mathsf{D}_{x,y} B(x,y) \qquad \Gamma, \mathsf{Q}_y B(b,y) \vdash A}{\Gamma \vdash A}. \tag{DE}$$

Here b is fresh for Γ, A, and the passive parameters of B. The displayed local assumption is discharged.

Deterministic paired introduction. Put $\eta_B(z) = z$ for a literal equation B, and $\eta_B(z) = 0$ for a compound B. Then

$$\frac{\Gamma, B(t, \eta_B(z)) \vdash B(t, Sz)}{\Gamma \vdash \mathsf{D}_{x,y} B(x,y)}. \tag{DI}$$

The parameter z is fresh for Γ, t, and the passive parameters of B. The local assumption is discharged. The rule has no independent case/progress certificate flag: the matrix root fixes the choice.

There are no further primitive logical, equality, successor, coverage, or induction axioms. Weakening, assumption cut, and fresh renaming are admissible. The presentation is the consequence relation of this auxiliary calculus on public contexts and conclusions. The neutral-resource code change adds no mathematical rule to this inventory.

3 Why the inherited theory has the required strength

The complete inherited proof is supplied as both `reference/guarded_source_report.pdf` and its source. It includes the earlier scalar arithmetic, logical, and coding arguments with attribution. This section identifies the dependency precisely and gives the short parts relevant to the present change. Reusing an established equivalence is different from claiming to have newly formalized it.

3.1 Soundness, including auxiliary induction

Use the following auxiliary interpretation into iPRA:

$$\begin{aligned}(s = t)^* &= (s = t),\\ (\mathsf{Q}_y B)^* &= B^*(0) \to \forall y\, B^*(y),\\ (\mathsf{D}_{x,y} B)^* &= \begin{cases} \exists x\, \forall y\, (B(x,y) \to B(x,Sy)), & B \text{ an equation},\\ \exists x\, (B^*(x,0) \to \forall y\, B^*(x,y)), & B \text{ compound}. \end{cases}\end{aligned}$$

Numerical substitution preserves a matrix's root constructor. Therefore the interpretation commutes with numerical substitution.

The computation schemas, hypothesis, and both branches of SE are sound. Atomic DI proves the indicated progressive statement directly. Compound DI is sound by zero-or-successor coverage. In atomic DE, equation induction turns progress into the conditional closure exported by Q. Compound DE is ordinary existential elimination. These observations give soundness by induction on the finite derivation.

For a guarded atomic $E(x, y)$, computation proves $E(x, 0)$. Consequently the progressive and conditional readings agree, by equation induction in one direction and the true zero instance in the other. Structural induction on public formulas proves

$$\mathsf{iPRA} \vdash A^* \leftrightarrow A^\circ.$$

This agreement is needed only for the public context and conclusion.

3.2 The completeness bootstrap

Auxiliary atomic DI, DE, and the zero branch of SE already derive contextual equation induction:

$$\frac{\Gamma \vdash E(0) \qquad \Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash E(r)}.$$

To see this, introduce $\mathsf{D}_{x,y} E(y)$ using an unused witness coordinate x, eliminate it to obtain $\mathsf{Q}_y E(y)$, and apply the base $E(0)$ at r. No equality repair or ordinary logical macro is needed for this step.

The inherited equational dependency theorem then restores transport whose source is literal zero. Its proof uses equation induction, computation, and transport with permitted sources. In particular it does not presuppose unrestricted substitution into a restricted derivation. After numerical equality is restored, an external induction on public templates restores transport through public paired formulas.

The guarded padding construction consequently recovers the previously established native scalar calculus on public formulas. That calculus reconstructs all intuitionistic first-order connectives, numerical coverage, separation, and quantifier-free induction. The scalar bridge constructs a predecessor, reflection of equality into terms, pairing, and a compiler for each primitive recursive definition. These inherited steps yield the reverse interpretation of ordinary iPRA.

Theorem 3.1 (Inherited equivalence, transferred unchanged)**.** *The public calculus of Section 2 is bi-interpretable and equiconsistent with full first-order iPRA. Both numerical domains are whole, numerical equality is actual equality, and both numerical composites are identified with the identity by the graph $x = y$. Changing the administrative neutral encoding from an empty list to scalar zero preserves this public theory and all these conclusions.*

Proof. For the first assertion, use the public consequence and formula-composite theorems of the inherited guarded-source report, composed with the native scalar/iPRA bridge. In the scalar-to-iPRA direction, translate each fixed iterator by its primitive recursive definition. In the reverse direction, compile primitive recursive definitions into scalar terms using the inherited pairing and recursion construction, and translate the ordinary logical connectives by the native and guarded macros.

The term round trips are proved from computation and equation induction; the formula round trips are the stated inherited logical composites. Neither direction restricts the numerical domain or replaces equality by a quotient relation. The contradiction is $0 = 1$ in the scalar presentation and maps to the usual arithmetic contradiction. Hence the interpretations also give equiconsistency.

The final assertion is proved by the neutral-resource invariant in Section 4. Public terms, formulas, and inference patterns are unchanged. Thus the interpretation maps and their composite proofs are unchanged as well. □

The existing Lean development proves the native scalar/iPRA bridge. The incoming guarded-source extension is a written mathematical proof; this work has not supplied a new Lean or Rocq verification of it. The selected Lean files in the archive are references, not a complete build environment. The existing Lean project was not rebuilt during this work.

4 The neutral-resource reduction

4.1 Certificate interface

Each compact checker reads a finite JSON array. Its final item is the goal; preceding items are instruction packets consumed from right to left. A used packet has four fields

$$[m, s, u, t].$$

The major premise m is recursively checked before it can supply an assumption. An empty major in the old local convention is a neutral truth resource used by computation and introduction packets. The new convention spells that resource as the scalar integer zero.

Formula arities 1, 2, and 3 encode D, equality, and a Q resource. A canonical Q resource is a triple of identical matrix encodings. Canonical numerical zero is also scalar zero, but occurs in the numerical sort, not at formula depth. The arity-aware traversal distinguishes these roles. A unary numerical array encodes successor, and a ternary numerical array encodes the iterator. Protected binding slots and local eigenparameter floors implement capture-avoiding opening and freshness.

The compact contract takes a canonical public statement as its proposed conclusion. The checkers can accept additional auxiliary goals. The strict profile explicitly checks the public goal grammar and its scope. It does not additionally impose exact integer/list typing on every unused value in the whole input. That stronger interface is a distinct contract.

Proposition 4.1 (Neutral resource replacement)**.** *The scalar-neutral variants are sound for the same public theory and complete under the same canonical-public-statement convention as their empty-list-neutral predecessors.*

Proof. At formula depth, administrative scalar zero denotes truth. Replacing the initially retained administrative truth by this spelling preserves the successful-trace soundness invariant: each retained

item is a decoded assumption, a computation theorem, or administrative truth. The arithmetic supplies, matching of elimination majors, local floors, and deterministic introduction calculation are otherwise identical.

For completeness, use the inherited well-sorted producer and emit each neutral major as zero. It closes against the new initial truth. Both zero and the old empty list are falsy, so the ensuing computation or introduction branch is unchanged. The non-neutral majors, public formulas, numerical terms, and witness/template fields are unchanged. Thus every public theorem still has a certificate.

This is the neutral-resource argument already established in the reversal package, applied to the guarded-source kernel. It does not identify the sets of all accepted raw JSON strings. In particular, stale certificates with the old neutral major may be rejected. □

4.2 The actual conversion is deliberately narrow

The supplied untrusted converter changes only a four-field command's major-premise field when that field is exactly `[]`:

```
if isinstance(command, list) and len(command) == 4 and command[0] == []:
    command[0] = 0
```

It does not replace every empty list recursively. In particular a falsy introduction-mode field is preserved, as are all numerical terms and public formulas. The audit checks every positive frozen producer packet for unexpected empty-list positions. The conversion claim concerns this well-sorted certificate interface, not arbitrary malformed input.

4.3 The complete byte-profile program

The following is the entire trusted program, including input and verdict. It imports only the standard JSON library. The normalized module has 521 UTF-8 bytes; presentation line wrapping below does not change that file.

```
import json

def c(a, g=[0], k=0):
    i = lambda a, t=c, d=0: [i(b, t, d + 5 - len(a)) for b in a] if a * 0 == [] else a if k <= a < d else d %
     2 and i(t, d=3)
    if i(a) in g:
        return
    m, s, u, t = q()
    a = m == ([s, u] if s else [t] * 3) and i(t, u) == a and i(t, s) or a
    h = c(m, g, k) or m * 3 or (u and [[s and [s], u, t], i(t, [s, u, t], 3) if s else u]) or (a := i(*a, t,
     2))
    k -= 1
    c(i(a, [k]), g + [i(h, (m or len(a) == 2) and k)], k)
q = json.load(open(0)).pop
c(q())
print('ACCEPT')
```

Acceptance requires exit status zero and exactly `ACCEPT` followed by one newline. Exceptions, nontermination, and resource exhaustion are not acceptance. As with the baseline, mathematical completeness uses ideal unbounded execution; a physical Python interpreter has resource limits. No producer, external theorem database, or auxiliary validator module is imported by the checker.

5 Measurements and executable evidence

All coordinates charge a whole standalone module. Normalize by `ast.unparse(ast.parse(source))` plus one LF under CPython 3.12.14. The measured checker modules contain no docstrings. Significant-token counts exclude comments, whitespace, indentation, and stream markers; AST counts include every node yielded by `ast.walk`. Gzip uses level 9 and `mtime=0`, including the gzip wrapper, with zlib 1.3.2. It is an informational coordinate.

Complete module	Tokens	AST	Bytes	Gzip-9
Incoming byte profile	264	301	522	328
Scalar-neutral byte profile	263	300	**521**	327
Incoming AST profile	261	279	603	345
Scalar-neutral AST profile	260	**278**	602	344
Incoming strict public profile	490	561	1202	571
Scalar-neutral strict public profile	489	560	1201	571

Keeping the unchanged incoming 257-token program, the combined coordinate envelope is

257 tokens / 278 AST nodes / 521 bytes.

Different programs attain these coordinates. In particular the 521-byte program does not have 257 tokens or 278 AST nodes. Its normalized-byte improvement over the actual 522-byte baseline is approximately 0.192%. The strict public program is only compared with its identical-grammar 1202-byte predecessor. Other incoming programs with different public grammars already have smaller strict interfaces; no global strict record is claimed.

5.1 What was tested

The reproduction harness executes 258 complete-process expectations: 21 fixture families, original and converted certificates, three original and three new checker profiles, ordinary Python and `python-O`, and six additional stale-neutral-spelling controls. Of the 258 expectations, 136 require acceptance and 122 require rejection. There were no mismatches and no timeouts.

Positive fixtures cover both computations, reflexivity, guarded atomic introduction and elimination, compound case introduction, implication elimination, induction with a used local hypothesis, and derived transport with a literal-zero source. Two sound auxiliary goals are accepted by the compact programs and deliberately rejected by the public-goal validator. Negative fixtures cover $0 = 1$, incorrect computation endpoints, missing instructions or fields, invalid goals, obsolete atomic case introduction, context leakage, future eigenparameter capture, and stale neutral spelling.

The exact per-process receipts and program hashes are supplied. These finite checks support the implementation comparison. The universal soundness and equivalence claims depend on the written invariants and inherited mathematical proofs, not on the number of successful tests.

6 An exact criterion for conditional and progressive closure

This section isolates a mathematical fact useful for further reductions. It is proved here; no literature-wide priority claim is made. Its role is to identify what a proposed replacement of case introduction by progress must actually preserve.

Let $E(y, \vec{a})$ be a literal equation, with the parameters $\vec{a}$ fixed. Write

$$B := E(0, \vec{a}), \qquad C(E) := B \to \forall y\, E(y, \vec{a}), \qquad P(E) := \forall y (E(y, \vec{a}) \to E(Sy, \vec{a})).$$

All following proofs are intuitionistic. Only equation induction is used.

Lemma 6.1 (Anchored progress)**.** *In iPRA,*

$$P(E) \to C(E), \qquad C(E) \leftrightarrow (B \to P(E)).$$

Proof. Assume $P(E)$ and B. Equation induction on E gives all its instances, proving the first implication. For the second assertion, $C(E)$ together with B yields $\forall y\, E(y)$, which immediately gives $P(E)$. Conversely, assume $B \to P(E)$ and then B. Obtain $P(E)$ and apply the first implication with the retained B. Discharge B. □

Theorem 6.2 (Sharp agreement criterion)**.** *Suppose the base $B = E(0, \vec{a})$ is decidable. Then*

$$(C(E) \leftrightarrow P(E)) \quad \leftrightarrow \quad (\neg B \to P(E)). \tag{1}$$

The hypothesis holds for equations in iPRA. More generally the result holds for any predicate with decidable base for which the indicated induction instance is available.

Proof. First suppose $C(E) \leftrightarrow P(E)$ and $\neg B$. From a hypothetical B one obtains contradiction, and hence every $E(y)$ by explosion. Thus $C(E)$ holds; the assumed equivalence gives $P(E)$.

Conversely suppose $\neg B \to P(E)$. The implication $P(E) \to C(E)$ has already been proved. To prove $C(E) \to P(E)$, assume $C(E)$ and split by the decidability $B \vee \neg B$. In the B branch, $C(E)$ gives all instances of E and hence $P(E)$. In the $\neg B$ branch use the assumed implication. Discharge $C(E)$.

The only induction used is the literal-equation instance in the previous lemma. Decidability is used at one explicit split, not as unrestricted excluded middle. □

6.1 Consequences and examples

The syntactic guard (Guard) makes B a theorem, so it satisfies (1). A true zero instance is sufficient, but not necessary: a predicate already closed under successor also satisfies it. For example $E(y) \equiv (0 = 1)$ is false independently of y. Both $C(E)$ and $P(E)$ are provable intuitionistically. The criterion therefore describes a strictly broader semantic family than true-base guards.

The tempting unrestricted replacement fails at

$$E(y) \equiv (y = 1).$$

Its base is false, so iPRA proves $C(E)$. Its progressive statement would take $E(1)$ to $E(2)$, contradicting $2 \neq 1$. Thus iPRA proves $\neg P(E)$. This is a separation inside arithmetic, not merely a finite-model suggestion. The same obstruction is already recognized in the inherited guarded work; the contribution here is the exact criterion.

6.2 Existential witnesses require a separate qualification

For $E(x, y)$ define C_x and P_x with x fixed. Pointwise agreement $\forall x(C_x \leftrightarrow P_x)$ implies

$$\exists x\, C_x \leftrightarrow \exists x\, P_x.$$

The converse need not hold. Different existential witnesses can conceal a pointwise failure.

For a concrete arithmetical example, let $C_{\mathrm{sel}}(x, a, b) := I(x, a; u.b)$, with u unused. It is a at $x = 0$ and b at successors. Set

$$E(x, y) \equiv C_{\mathrm{sel}}(x, 1, y) = 1.$$

At $x = 0$ this equation is always true, so both existential closures are provable with witness zero. At $x = 1$ it reduces to $y = 1$, where the pointwise readings differ. Consequently the pointwise criterion must not be advertised as a necessary condition for agreement after existential quantification. The correct existential analogue permits changing the witness:

Theorem 6.3 (Exact criterion after existential binding)**.** *Let $D_C := \exists x\, C_x$, $D_P := \exists x\, P_x$, and $F := \exists x\, \neg E(x, 0)$. Then iPRA proves*

$$(D_C \leftrightarrow D_P) \quad \leftrightarrow \quad (F \to D_P).$$

Proof. Equation induction gives $D_P \to D_C$. If $D_C \to D_P$ and F hold, open a witness to the false base. Its conditional closure follows by explosion, so D_C holds and hence D_P.

Conversely assume $F \to D_P$, and open a witness a to D_C. Decide the equation $E(a, 0)$. If it holds, its conditional closure gives every $E(a, y)$, and hence P_a and D_P. If it fails, a witnesses F and the assumed implication gives D_P, possibly with a different witness. Discharge the opened witness. No quantified formula is subjected to excluded middle. □

6.3 A base-retaining normalization

There is a useful exact normalization which also explains why a semantic improvement need not shorten a checker. With $B = E(0)$, put

$$A(y) := B \wedge E(y).$$

Then

$$C(A) \;\leftrightarrow\; P(A) \;\leftrightarrow\; C(E). \tag{2}$$

Indeed, $A(0) \leftrightarrow B$, and its conditional closure is equivalent to $B \to \forall y\, E(y)$. If $C(E)$ and $A(y)$ hold, the retained B gives $E(Sy)$, hence $A(Sy)$; thus $P(A)$ follows. Conversely, under B, $P(A)$ gives the progress step for E; equation induction yields every $E(y)$. This proves (2) without needing a decision of B in this particular argument.

The normalized predicate can again be made a literal equation. If $E(y)$ is $s(y) = t(y)$ and $\langle \cdot, \cdot \rangle$ is a proved injective primitive recursive pairing function, use

$$\widehat{E}(y) := \langle s(0), s(y) \rangle = \langle t(0), t(y) \rangle.$$

Injectivity makes this equivalent to $A(y)$. Thus the normalization is available within iPRA and, after its arithmetic bridge, within the scalar theory. Its zero instance need not be true. If it is false, every instance is false, which makes progress vacuous.

This is a proved normal form, not a new measured kernel. Using it in a primitive presentation would require deriving pairing and its injectivity without a circular appeal to the sought completeness theorem, and charging the term expansion or the new constructor and its axioms. Also, the direct rule retaining both $E(0)$ and $E(z)$ is already the older anchored introduction from which the repository's Choice-DI system was derived. That rule is not a new result of this investigation.

6.4 Why the criterion does not itself produce a smaller checker

Checking the semantic condition $\neg E(0) \to P(E)$ would require a proof or an adequately specified syntactic class. It cannot be silently treated as a constant-cost parser test. The existing true-base guards give a simple checkable sufficient condition. Replacing them by the full semantic criterion without counting proof certificates and their verification would hide complexity outside the trusted program.

For supplementary verification, `closure_criterion.py` checks all 8190 Boolean sequences of lengths 1 through 12, treating progress as closure along each finite sequence. The code checks $P \to C$, $C \leftrightarrow (B \to P)$, and $(C \leftrightarrow P) \leftrightarrow (\neg B \to P)$, and records the false–true–false counterexample. It does not test the existential theorem or the pairing normalization. These finite checks do not replace the intuitionistic proofs above.

7 Further research questions

1. **A genuine arithmetic reduction.** Can a numerical constructor or binding operation be removed while the entire equivalence proof and complete checker both become simpler? The existing successor-seeded, self-seeded, and register presentations should be comparison points. An initialized iterator compiler cannot be assumed before the necessary predecessor and selector have been derived.

2. **A better checkable class of matrices.** Can the condition in (1) be realized by a syntax simpler to validate than the current guarded grammar? A candidate must include both the proof of full public expressiveness and the cost of checking its grammar. Semantic agreement alone is not a small axiom recognizer.

3. **Unrestricted ordinary formula readings.** Can deterministic introduction coexist with the full original Eq/Q/D language and its ordinary readings, with fewer complete-checker resources than the existing witness-selected rule? The counterexample $y = 1$ rules out the direct progress-only replacement, not every possible redesign.

4. **Scope represented by proof binders.** A tree certificate could allocate opaque eigenparameters when entering a binder, preventing raw input from naming them. This may simplify freshness proofs. The additional tree opening and opaque-atom handling must be counted; no size improvement for this architecture was verified in this investigation.

5. **Mechanize the current best public/auxiliary boundary.** The native bridge is already formalized. Formalizing the guarded-source extension and its neutral-resource decoding would close the most relevant verification gap without re-proving the entire arithmetic development.

8 Reproduction, attribution, and status

From the extracted archive directory:

```
python3 reproduce.py
python3 closure_criterion.py
python3 code/checker_bytes.py < tests/zero/guarded_induction_with_used_hypothesis_local.json
pdflatex -interaction=nonstopmode -halt-on-error ipra_compact_update.tex
pdflatex -interaction=nonstopmode -halt-on-error ipra_compact_update.tex
```

The Python reproduction uses only the standard library. The main TeX source is self-contained and uses ordinary TeX Live packages. No external figure, network call, shell escape, or bibliography processor is needed. The frozen inherited report has its own checker listing under `reference/code`; compile it from the `reference` directory.

The scalar arithmetic, native logical macros, guarded public grammar, source restriction, deterministic introduction, arity codec, and scalar neutral-resource idea are all inherited from ProveIt and credited above. The new deliverable combines the last encoding with the latest checker, audits the true current baseline, proves the closure criterion explicitly, and supplies a reproducible comparison. The upstream MIT-0 license is included.

The archive's `CLAIMS.json` distinguishes the measured implementation result, inherited mathematical equivalence, new written analysis, and unachieved stronger goal. `SHA256SUMS` records the delivered files before a reproduction run rewrites generated receipts or PDF metadata. No remote repository changes were made. There is no claim of an internal consistency proof for iPRA, an absolute source-size lower bound, or a new proof-assistant-checked equivalence.

References

[1] Vladimir Reshetnikov, *ProveIt: Concise presentations of intuitionistic primitive recursive arithmetic*, README and Lean development, inspected at commit `a4e5e96c456303cf231eab2aeeb3291d8889833f`. https://github.com/VladimirReshetnikov/ProveIt/tree/a4e5e96c456303cf231eab2aeeb3291d8889833f/Logic/PrimitiveRecursiveArithmetic/ConciseIPRA.

[2] ProveIt, *A guarded-source presentation of intuitionistic primitive recursive arithmetic*, incoming archive `ipra_guarded_source_2026-11-11.zip`, at the inspected commit. The complete delivered report, source, and exact comparison modules are included under `reference/`. https://github.com/VladimirReshetnikov/ProveIt/tree/a4e5e96c456303cf231eab2aeeb3291d8889833f/Logic/PrimitiveRecursiveArithmetic/ConciseIPRA/docs/incoming.

[3] ProveIt, *A smaller checked presentation of iPRA*, incoming archive `ipra_reversal_core_2026-11-11.zip`, section "Scalar zero as a neutral formula." Its exact standalone TeX source is included as `reference/reversal_report.tex`. https://github.com/VladimirReshetnikov/ProveIt/blob/a4e5e96c456303cf231eab2aeeb3291d8889833f/Logic/PrimitiveRecursiveArithmetic/ConciseIPRA/docs/incoming/ipra_reversal_core_2026-11-11.zip.